Centre of Gravity (COG) — User Guide

Depot-location solver · handover documentation · 31 July 2026

1. What this tool does

COG is a desktop tool that finds the best depot locations from delivery demand data. Give it a CSV of delivery points — latitude, longitude, parcel volume — and it computes depot locations that minimise total demand-weighted travel distance, then opens an interactive map report in your browser.

It supports three ways of framing the siting problem:

- **Single Depot** — where should the one depot go?
- **Multi Depot** — where should k depots go, and which customers does each serve?
- **Fixed + Variable Depots** — some depots already exist (a leased warehouse, a site you are committed to); where should the *remaining* depots go, and optionally, how much demand can each existing site actually carry?

Every mode reports the same objective — total demand-weighted straight-line distance — so runs are comparable to each other. COG's own accuracy has been independently checked against two other tools (ArcGIS, QGIS) on a 69,743-point dataset; see the companion **Validation Report** for the evidence. This guide covers how to run the tool and how to read its output.

2. Installation

Requires *uv* (https://docs.astral.sh/uv/). From the project folder:

```
uv sync
```

Optional: `uv sync --extra webview` also installs `pywebview`, which lets the report open inside an embedded window instead of your default browser.

3. Running the app

```
uv run cog
```

This opens the desktop window. The workflow is always the same four steps:

1. **Browse** to your input CSV (or type/paste the path directly).
2. Pick a **Solver Mode** — the three pills at the top of the input card — and fill in that mode's parameters (Section 5 below covers each mode).
3. Check the **Output Folder** — it defaults to `outputs/` under the folder you launched the app from; **Browse** to change it.
4. Click **Run Solver**.

While solving, the button reads "Running…", a progress bar tracks the multi-start search, and the status line below it names the current step (e.g. "Multi-start seed 4 of 10…"). The window stays responsive throughout — solving runs on a background thread.

When a run finishes:

- The **Results** tab fills with the depot table — one row per depot, with its coordinates, assigned demand and point count — plus the objective summary line and, if you set a capacity cap, a cap note.
- The **Log** tab records a timestamped line for each step, including the exact path of the saved report.
- An HTML map report opens automatically in your browser (or embedded window, if you installed the `webview` extra).

Each run writes its own file, so successive runs never overwrite one another:

```
outputs/clusters_map_<dataset>_<mode>_<timestamp>.html
```

e.g. `outputs/clusters_map_malaga_pdc_multi_20260721_153045.html`. The tag is `single`, `multi`, or `fixed`.

4. Input file format

A CSV file with these columns (any extra columns are ignored):

column	meaning
`latitude`	decimal degrees
`longitude`	decimal degrees
`demand_items`	parcel volume for that point

Two things worth knowing before you load a file:

- **Points with zero or negative demand are dropped**, silently, before solving. They do not count toward any depot's assigned demand or point total, and they do not influence where a depot is placed. If your row counts don't match what you expect, check for zero-demand rows first.
- **Capacity caps require whole-number demand.** If you set a capacity limit on any fixed depot (Section 5.3), every point's `demand_items` must be an integer, or the run fails with an explicit error before solving. Without a cap, fractional demand is fine everywhere.

Example files are in `inputs/`.

5. Solver modes in detail

5.1 Single Depot

No extra parameters. One depot is placed at the demand-weighted geometric median of every point — the location that minimises the sum of demand-weighted distances to all of them.

5.2 Multi Depot

One parameter: **Number of depots (k)**. Ten independent starts are run, each starting from a full demand-weighted k-means fit (itself k-means++ seeded) and then refining that result by *alternating location-allocation* (Section 6) — the start with the lowest objective wins. Use this when no depot already exists and you want the best possible siting for a given depot count.

5.3 Fixed + Variable Depots

Use this when one or more depots are already committed — an existing warehouse, a site you've already leased — and you want COG to place any *additional* depots optimally around them.

Parameters:

- **Number of fixed depots** — 1, 2, or 3. Selecting a count reveals that many coordinate rows.
- For each fixed depot: **lat** / **lon** — its real-world coordinates, held fixed for the entire run.
- **Number of free depots** — how many additional depots COG should place. `0` is valid: every point is then simply assigned to its nearest fixed depot, with no optimisation.

Optional per-depot capacity. Tick **Constrain capacity** next to any fixed depot to reveal a **max demand (items)** field — a hard ceiling on the total demand (in items, not points) that depot may carry. Leave it unticked for no limit. Free depots are always unlimited; capacity only ever applies to fixed depots, because they are the ones tied to a real, finite site.

What happens under the hood when a cap is set:

1. Points are assigned to their nearest depot first, same as any other run.
2. If that leaves every capped depot under its limit, nothing else happens — this is the common case.
3. If a capped depot would be *over* its limit, COG re-solves the assignment with a min-cost-flow network instead: it finds the assignment that respects every capacity exactly while minimising total demand-weighted distance. This solve can, in principle, split a single point's demand across more than one depot to hit the capacities exactly. COG then *repairs* every split point by moving it wholly onto whichever depot took the largest share of it — and that repair step does **not** re-check the cap afterwards. Moving a split point's full demand onto its majority depot can push that depot marginally over its cap; MCF only guarantees the cap for the (possibly split) flow it solved, not for the whole-point assignment that replaces it. The Log tab and terminal report how many points needed this repair (`MCF split-repair: last=… max=… calls=…`). Zero is the usual result. A nonzero count means at least one point's demand was split and then moved whole, which can leave that depot marginally above its cap — so when the count is nonzero, check the assigned-demand column in the depot table against the caps you set, rather than assuming the cap held.
4. If there are no free depots and every fixed depot is capped, COG checks feasibility before solving at all: if the caps' total is less than the dataset's total demand, it stops with an explicit "Infeasible capacity" error rather than producing a silently wrong answer.

What a capped run looks like on the map. A cap makes some points travel further than they would otherwise. Once a depot is full, points near its boundary are served by a depot that is further away, so territories interlock rather than forming clean neighbouring blocks, and a few points can end up noticeably far from the depot serving them. This is expected, not a sign of a bug — it is the lowest-total-distance

assignment achievable under the caps (subject to the split-repair caveat above). On a run with four depots and caps of 5,000 and 10,000 items on two of them, 12.6% of points ended up served by a depot that was not their nearest, and the total rose from 428,739 to 471,236 item·km. If no territories interlock at all, the caps probably aren't binding.

6. How it works (methodology)

Projection. Coordinates are converted to local metres with an equirectangular projection centred on the demand-weighted mean of the dataset, so straight-line distances are accurate enough at city scale without the cost of a full geodesic calculation. This approximation is built for metro-scale problems, not continental-scale siting; accuracy degrades the further a dataset spans beyond a single metro area.

Weighted geometric median (Weiszfeld's algorithm). The core primitive used everywhere a depot needs to be placed: the point that minimises the sum of demand-weighted distances to a set of points. Start at the demand-weighted average, then repeatedly re-weight each point by `1/distance` and recompute, until the move is below tolerance. Single Depot mode is exactly one call to this.

Multi-start seeding + alternating location-allocation. Multi Depot and the free-depot part of Fixed + Variable use the same refinement loop, run from ten different starting points and keeping the best:

```
for seed in 0..9:
    start   = full demand-weighted k-means fit with this seed
              (k-means++ initialisation, run to convergence)
    result  = alternating location-allocation from that start:
                  repeat:
                      assign every point to its nearest depot
                      move each (non-fixed) depot to its cluster's
                        weighted geometric median
                  until the objective stops improving
    keep the seed with the lowest final objective
```

This is Cooper's alternating location-allocation algorithm. Each pass can only lower the objective, so it converges — but it converges to whichever *local* optimum is nearest its starting point. Ten different starting points sample ten different basins; keeping the best of the ten is materially better than trusting any single run, and is why the tool reports "seed X of 10" progress rather than a single instant answer.

Capacity-constrained assignment. Described in Section 5.3 — nearest-depot by default, min-cost-flow (unit cost = distance, flow = demand) only when a fixed depot's cap would otherwise be exceeded, with a repair step so every point still ends up wholly assigned to one depot in the final report.

7. Reading the output

The depot table (Results tab). One row per depot: coordinates, assigned demand, point count, and — for capacity runs — the cap that applied, if any. Below the table, the objective line:

```
Objective: 422,725.8 item·km total, average 5.96 km per demand item
```

Lower is better. This is the number to compare between runs of the same dataset — e.g. k=4 vs k=5, or capped vs uncapped.

The HTML map report. Every point, coloured by assigned depot, on a Leaflet map — click a depot in the sidebar to isolate its cluster, click again to deselect, or select several to see their clusters combined; "Show all" clears the selection. Four stat cards along the top give depot count, point count, total demand, and the largest depot's share. A collapsible **Methodology** panel at the bottom explains exactly which algorithm produced this specific run. This report needs an internet connection to render the map background (Leaflet and the basemap tiles load from a CDN) — the depot numbers and sidebar are self-contained and work offline; only the map tiles need the network.

8. Validation & confidence

COG's placements have been independently checked: the same 69,743-point dataset was solved by two other tools (ArcGIS, QGIS) and every tool's depot coordinates were re-scored under one shared metric. QGIS lands within 0.9–4.3% of COG's cost at every depot count tested; ArcGIS within 3.4–15.7% while solving a more constrained, more realistic version of the problem (routing on real roads, from a fixed list of candidate sites). Full method, numbers, and the three-tool map comparison are in the companion **Validation Report** (`docs/COG_Validation_Report.md` / `COG_Validation_Report.pdf`) — read that document for evidence of correctness; this guide is about how to operate the tool.

9. Limitations — know these before relying on a result

- **Straight-line distance, not road distance.** Every objective and every depot placement optimises demand × straight-line kilometres. Real road-network travel will differ, sometimes substantially near coastlines, rivers, or limited-access geography.
- **Metro-scale only.** The equirectangular projection is built for single-metro datasets; accuracy degrades as a dataset spans further beyond that, so don't use this for national or continental siting.
- **Capacity requires integer demand.** Set a cap with fractional `demand_items` in the file and the run fails fast with an explicit error, rather than producing a capacity that doesn't mean what it says.
- **The map needs internet.** The depot table and objective work fully offline; only the map's basemap tiles need a connection.
- **Ten starts, not exhaustive search.** Multi-start local search is a strong practical method, not a global-optimum guarantee. Two runs of the exact same inputs will return the same answer (the ten seeds are fixed), but a fundamentally different starting strategy could in principle do better on an unusual dataset.

10. Troubleshooting

Symptom	Likely cause / fix
"Please select a CSV file" / "Please select a solver mode"	A required field was left empty before clicking Run Solver.
"CSV file missing required column"	The file is missing `latitude`, `longitude`, or `demand_items` — check column names exactly.

Symptom	Likely cause / fix
"File not found"	The CSV path is stale (moved/renamed file, or a relative path from a different working directory).
"Max demand (items) for fixed depot N is required when 'Constrain capacity' is checked"	The capacity checkbox was ticked but the max-demand field was left blank.
"Infeasible capacity: fixed depot demand caps sum to … < … total demand"	With zero free depots, every fixed depot's cap must add up to at least the dataset's total demand — raise a cap, or add a free/uncapped depot to absorb the rest.
"Demand must be integer-valued for … capacity"	A capacity cap is set somewhere and at least one `demand_items` value is fractional. Round/aggregate demand first, or remove the cap.
Map report opens but the background is blank/grey	No internet connection — the map tiles load from a CDN. The depot data itself is unaffected.

11. For maintainers

This guide covers day-to-day use. For the codebase itself:

- `README.md` — the same quick-start, kept in sync with this guide.
- `CHANGES.md` — the full history of *why* the code looks the way it does, written for whoever touches it next.
- `src/cog/solvers.py`, `geo.py`, `dataio.py`, `report.py`, `app.py` — one file, one job (see `CHANGES.md`'s refactor entry for the reasoning).
- `tests/` — golden-snapshot tests pinning solver behaviour; run with `uv run pytest` before and after any change to `solvers.py`.